



◆ OFFENSIVE SECURITY SERIES ◆

iOS STATIC ANALYSIS

IPA Structure, Info.plist, class-dump & Binary Secrets

class-dump · Info.plist · Entitlements · otool · strings · Hopper

class-dump

Info.plist

Entitlements

Binary Strings

AWS Keys

ASLR

Canaries

RedTeamGarage (RTG) | Offensive Security Education

<https://www.redteamgarage.com/>

LEGAL DISCLAIMER

This guide is produced by RedTeamGarage (RTG) for educational purposes only.
All techniques must ONLY be used on systems you own or have explicit written
authorisation to test. RTG assumes zero liability for any misuse of this material.

**TABLE OF CONTENTS**

iOS STATIC ANALYSIS — IPA Structure, Info.plist, class-dump & Binary Secrets

01.	INFO.PLIST + ENTITLEMENTS ANALYSIS
02.	CLASS-DUMP + BINARY STRING SECRETS
03.	iOS STATIC ANALYSIS CHEAT SHEET



01 — INFO.PLIST & ENTITLEMENTS

ATS + URL Schemes + Debugger

```
Info.plist + Entitlements Analysis

# Extract and inspect IPA structure
(kali)-[~]
kali $ unzip -q com.target.bankingapp.ipa -d ipa_extracted/
(kali)-[~]
kali $ ls ipa_extracted/Payload/BankingApp.app/
BankingApp      Info.plist      embedded.mobileprovision
_CodeSignature/ Assets.xcassets/ Frameworks/
Base.lproj/     GoogleService-Info.plist

# Check Info.plist for ATS and URL schemes
(kali)-[~]
kali $ plutil -convert xml1 ipa_extracted/Payload/BankingApp.app/Info.plist -o -
<key>NSAppTransportSecurity</key>
<dict>
  <key>NSAllowsArbitraryLoads</key>
  <true/>      ← ALL HTTP ALLOWED! ATS disabled
</dict>
<key>CFBundleURLSchemes</key>
<array>
  <string>banking</string>      ← custom URL scheme
  <string>bankingapp</string>
</array>

# Check entitlements
(kali)-[~]
kali $ codesign -d --entitlements :- ipa_extracted/Payload/BankingApp.app/BankingApp
<key>keychain-access-groups</key>
<array><string>TEAM123.com.target.bankingapp</string></array>
<key>com.apple.security.get-task-allow</key>
<true/>      ← DEBUGGER ATTACH ALLOWED!
```

Figure: ATS disabled (all HTTP allowed), debugger attach permitted, custom URL schemes found



02 — CLASS-DUMP & STRINGS

Method Headers + Hardcoded Secrets

```
class-dump + Binary Strings Analysis

# Extract ObjC class headers with class-dump
(kali)-[~]
kali $ class-dump -H ipa_extracted/Payload/BankingApp.app/BankingApp -o headers/
(kali)-[~]
kali $ ls headers/ | grep -iE "auth|crypto|network|login"
AuthManager.h
CryptoUtils.h
LoginViewController.h
NetworkManager.h

(kali)-[~]
kali $ cat headers/AuthManager.h
@interface AuthManager : NSObject
- (BOOL)loginWithUsername:(NSString *)username password:(NSString *)password;
- (NSString *)getStoredToken;
- (void)storeToken:(NSString *)token;
- (BOOL)verifyPIN:(NSString *)pin;
- (BOOL)checkBiometric;
@end

# Search binary strings for hardcoded secrets
(kali)-[~]
kali $ strings ipa_extracted/Payload/BankingApp.app/BankingApp | grep -iE "key|secret|password|api|token|http"
https://api.target.com/v2/
https://admin.internal.target.com/
sk-live-Abc123XYZdef456GHIjkl789
jwt_signing_secret_2024_prod
AIzaSyDaGmWka4JsXZ-HjGw6NXKXzEnSGS2K7JA
```

Figure: class-dump reveals AuthManager methods to hook, strings finds API keys in binary



03 — CHEAT SHEET

Quick Reference

Extract IPA	<code>unzip -q app.ipa -d ipa_extracted/</code>
Read Info.plist	<code>plutil -convert xml1 Info.plist -o -</code>
Check ATS	<code>grep -A5 NSAppTransportSecurity Info.plist (NSAllowsArbitraryLoads: true = insecure)</code>
Find URL schemes	<code>grep -A5 CFBundleURLSchemes Info.plist</code>
Check entitlements	<code>codesign -d --entitlements :- BinaryName</code>
get-task-allow check	<code>codesign -d --entitlements :- Binary grep get-task-allow (true = debugger ok)</code>
class-dump	<code>class-dump -H BinaryName -o headers/</code>
Find auth methods	<code>grep -rn "password\ token\ auth\ pin\ crypto" headers/</code>
Binary strings	<code>strings BinaryName grep -iE "http key secret password token"</code>
AWS key pattern	<code>strings BinaryName grep -E "AKIA[A-Z0-9]{16}"</code>
Firebase config	<code>cat GoogleService-Info.plist grep -A1 API_KEY</code>
Check ASLR	<code>otool -hv BinaryName grep PIE (MH_PIE = enabled)</code>
Check canaries	<code>otool -Iv BinaryName grep stack_chk</code>
Check cryptid	<code>otool -l BinaryName grep cryptid (0 = decrypted, 1 = still encrypted)</code>
Ghidra / radare2	<code>r2 -A BinaryName → afl (functions) → pdf @ sym.func</code>
React Native bundle	<code>cat main.jsbundle js-beautify > pretty.js && grep -iE "apiKey secret" pretty.js</code>



CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG MISSION

Real-world offensive security training - practical, no fluff, built for serious learners.

Build the most comprehensive red team technique library available anywhere.

Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.